Day 9 – Testing with pytest-django

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment active**: Ensure your terminal shows `(.venv)` in the prompt.
   - **Clean Environment**: Clean up any previous test runs.
   - **IDE Ready**: Open `pytest.ini`, `games/tests/test_webhook.py`, and `games/tasks.py` in your IDE.
2. **Prerequisites**:
   - You must have completed Day 8 successfully (Celery worker, Redis queue, and database logging are configured and working).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain unit testing philosophies and what to mock vs. what to test.
- Install and configure `pytest`, `pytest-django`, and `pytest-mock` packages.
- Define reusable database fixtures with pytest.
- Inject database permissions in tests using `@pytest.mark.django_db`.
- Mock external network requests and Celery delay queues using `@patch`.
- Run a test coverage report and read its results to find untested paths.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus
1. Hook & Big Picture	15 min	Define the value of automated testing. Discuss why we mock network and clock dependencies.
2. Key Concepts & Core Ideas	15 min	Explain pytest fixtures, transactional db isolation, mocks, patches, and code coverage.
3. Live Coding Walkthrough	45 min	Install testing plugins, write configuration files, write fixtures, build three custom tests, run tests, and inspect coverage.

4. Check for Understanding	10 min	Q&A on mocking boundaries, `side_effect` vs `return_value`, and coverage reports.
5. Class Wrap-up & Checkpoint	5 min	Run test command and verify all tests pass with ≥70% coverage.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

If we make a change to our order code on Day 12, how do we know we didn't break our Day 8 webhook logic? Do we start our web server, run Redis, spin up a Celery worker, and manually create an order every time? No, we write automated scripts that test all pathways in under 2 seconds.

- **Mocking External I/O**: Real tests shouldn't hit real external publisher servers, because if a publisher is down, our internal tests will fail. We use **mocking** to simulate the network response.
- **Q&A**:
  - **Question**: Why do we mock external I/O?
    - *Answer*: Real external publisher servers can be slow, rate-limited, or completely offline. If our test suite depended on them, our internal tests would fail due to external network factors. Mocking simulates the network response instantly and predictably, guaranteeing isolated, fast, and deterministic test runs.

2. Key Concepts & Core Ideas (15 min)

Introduce these testing concepts:

- **Unit vs Integration Testing**:
  - *Unit test* verifies a single function in isolation (mocking all external dependencies).
  - *Integration test* checks if multiple modules work together (like Django views talking to the database).
- **`@pytest.fixture`**: Reusable functions that setup state before a test executes and tear down state afterward.
- **`@pytest.mark.django_db`**: By default, pytest blocks database calls to keep tests fast and isolated. This decorator opens up a temporary database transaction which rolls back at the end of the test.
- **`@patch('module.name')`**: Replaces an import reference with a dummy `MagicMock` object for the duration of the test. **Rule**: *Always patch where the module is imported and used, not where it is defined.*
- **Code Coverage**: The percentage of code lines executed during testing. High coverage indicates that lines were executed, but does not guarantee bug-free logic.
  - **Question**: What does '70% coverage' actually mean (and not mean)?
    - *Answer*: It means 70% of the lines of code in our application were executed at least once during

the test suite run. It does *not* mean the code is 70% bug-free, nor does it guarantee that edge cases, exception handling, race conditions, or business logic correctness have been validated.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Package Installation & Configuration

- **Concept**: Install our testing libraries. We must tell `pytest` where our Django settings file is located using a local configuration file.
- **Code**: Install plugins:

```
uv add pytest-django pytest-mock
```

Create `pytest.ini` in the project root directory:

```
[pytest]
# Tell pytest where the Django settings module is defined
DJANGO_SETTINGS_MODULE = gamekey_platform.settings
# Match files starting with test_ as test scripts
python_files = test_*.py
```

Create an empty init file under `games/tests/` directory to mark it as a test suite:

```
games/tests/__init__.py
```

- **Verify**: Verify that running `pytest` from the terminal scans for tests, even if it finds zero.

Step 3.2: Writing the Fixtures

- **Concept**: We will create reusable data fixtures in `games/tests/test_webhook.py` that set up a publisher profile and an expired game key inside the temporary test database.
- **Code**: Create `games/tests/test_webhook.py`:

```
import pytest
from django.contrib.auth.models import User
from django.utils import timezone
from datetime import timedelta
from unittest.mock import patch, MagicMock
from django.core.management import call_command
from games.models import Publisher, Game, GameKey, WebhookDeliveryLog


# Define a reusable database fixture
# Passing the 'db' parameter tells pytest this fixture requires database access
@pytest.fixture
```

```
def publisher_user(db):
    user = User.objects.create_user(username='pub_user', password='pass')
    publisher = Publisher.objects.create(
        name='Test Publisher',
        webhook_url='https://example.com/webhook',
        webhook_secret='supersecret',
        user=user,
    )
    return publisher


@pytest.fixture
def expired_game_key(db, publisher_user):
    game = Game.objects.create(
        title='Epic Game',
        publisher=publisher_user,
        price='29.99',
    )
    return GameKey.objects.create(
        key_string='TEST-KEY-0001',
        game=game,
        status='active',
        expires_at=timezone.now() - timedelta(hours=1), # Set expiry in the past
        owner=publisher_user.user,
    )
```

- **Verify**: Ensure that these fixtures load without syntax errors.

Step 3.3: Writing the Test Logic

- **Concept**: Add three unit tests to verify:
  1. The expiration management command successfully detects expired keys and dispatches tasks to the Celery queue.
  2. The Celery task logs a successful result inside the audit table when the publisher's HTTP response is `200`.
  3. The Celery task logs a failure entry inside the audit table when requests raise connection errors.
- **Code**: Append the tests to `games/tests/test_webhook.py`:

```
@pytest.mark.django_db
# Patch the Celery task's delay queue. We replace it with a mock object
@patch('games.tasks.send_expiry_webhook_async.delay')
def test_management_command_dispatches_tasks(mock_delay, expired_game_key):
    # Run the Django management command programmatically
    call_command('check_expired_keys')

    # Assert that the Celery delay function was called
    assert mock_delay.called

    # Verify that correct kwargs parameters were passed to the task
    call_args = mock_delay.call_args
```

```
    assert call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'


@pytest.mark.django_db
# Patch the synchronous requests.post library function used in the task file
@patch('games.tasks.requests.post')
def test_webhook_task_logs_success(mock_post, publisher_user, expired_game_key):
    from games.tasks import send_expiry_webhook_async

    # Mock the HTTP response object to return status code 200
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_post.return_value = mock_response

    # Invoke the Celery task function synchronously
    send_expiry_webhook_async(
        publisher_id=publisher_user.id,
        game_key_str='TEST-KEY-0001',
        game_title='Epic Game',
        expired_at_iso=expired_game_key.expires_at.isoformat(),
        attempt=0,
    )

    # Verify that a successful WebhookDeliveryLog record was created in the DB
    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is True
    assert log.response_status == 200


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_failure(mock_post, publisher_user, expired_game_key):
    from games.tasks import send_expiry_webhook_async

    # Simulate a network crash by raising an exception
    mock_post.side_effect = Exception('Connection refused')

    # Check that the exception is raised correctly up to the task caller
    with pytest.raises(Exception):
        # Use .apply() to run task synchronously within test transaction context
        send_expiry_webhook_async.apply(kwargs=dict(
            publisher_id=publisher_user.id,
            game_key_str='TEST-KEY-0001',
            game_title='Epic Game',
            expired_at_iso=expired_game_key.expires_at.isoformat(),
            attempt=0,
        ))

    # Verify that a failure WebhookDeliveryLog record was created in the DB with the exception
    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is False
    assert 'Connection refused' in log.error_message
```

- **Verify**: Run `pytest` inside the terminal and ensure all tests run and pass.

- ⚠ **Common Pitfalls**:
  - **Wrong `@patch` target**: Note that patching `requests.post` globally does not work. We must patch `games.tasks.requests.post` because that's where the request is executed.
  - **Calling `.delay()` inside tests**: Calling `.delay()` would push the task onto a real Redis broker. For testing, we mock `.delay` (Test 1) or run the function synchronously using `task.apply(...)` (Test 3).

Step 3.4: Generating Coverage Reports

- **Concept**: Measure our test coverage to see what percentage of code lines were touched during execution.
- **Code**: Run coverage:

```
# Run pytest and collect coverage report on the 'games' directory, listing missing lines
pytest --cov=games --cov-report=term-missing
```

- **Verify**: Review the command output. Highlight any lines marked missing and explain how to write tests targeting them.

---

4. Check for Understanding (10 min)

> ***Question**: "Why do we `@patch('games.tasks.requests.post')` rather than `@patch('requests.post')`?"*
> ***Answer**: We patch where the module is imported and used, not where it is defined. In `games/tasks.py`, the code uses `requests.post`. If we patched `requests.post` globally, `games/tasks.py` would still use its local, unpatched reference to the real `requests.post` function.*

> ***Question**: "What does `mock_post.side_effect = Exception('Connection refused')` do differently from `mock_post.return_value = ...`?"*
> ***Answer**: `.return_value` makes the mock function return a specific value (like a mock response object) when called. `.side_effect` raises the specified exception when the mock function is called, allowing us to test how the code handles connection failures or timeouts.*

> ***Question**: "If a test has 100% line coverage, does that mean it's bug-free? Why or why not?"*
> ***Answer**: No, 100% line coverage only means every line of code was executed at least once during the tests. It does not verify different combinations of inputs, edge cases, race conditions, logical errors, or unexpected database states.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ Running `pytest` runs and passes all 3 tests.
  - ○ Running `pytest --cov=games --cov-report=term-missing` outputs a report showing test coverage of at least 70% for the custom app logic.